



natural language generation

LING 430 Computational Linguistics Fall 2026

lecture 7a

Oct 6 2026

Siyu Liang

siyu.liang@rice.edu



last time

- probability for language data
 - conditional probability, Bayes' theorem
- n-gram language models
 - training
 - evaluation
 - zeros and smoothing



agenda

- generation
- sampling
- n-gram (revisit)
- perplexity (revisit)
- temperature
- a tweet generator



today's notebook

class07a.ipynb

- course website → week 7 → activity
 - **File → Save a copy in Drive**
- Canvas → Files → tweets_trump.txt



generation

natural language generation

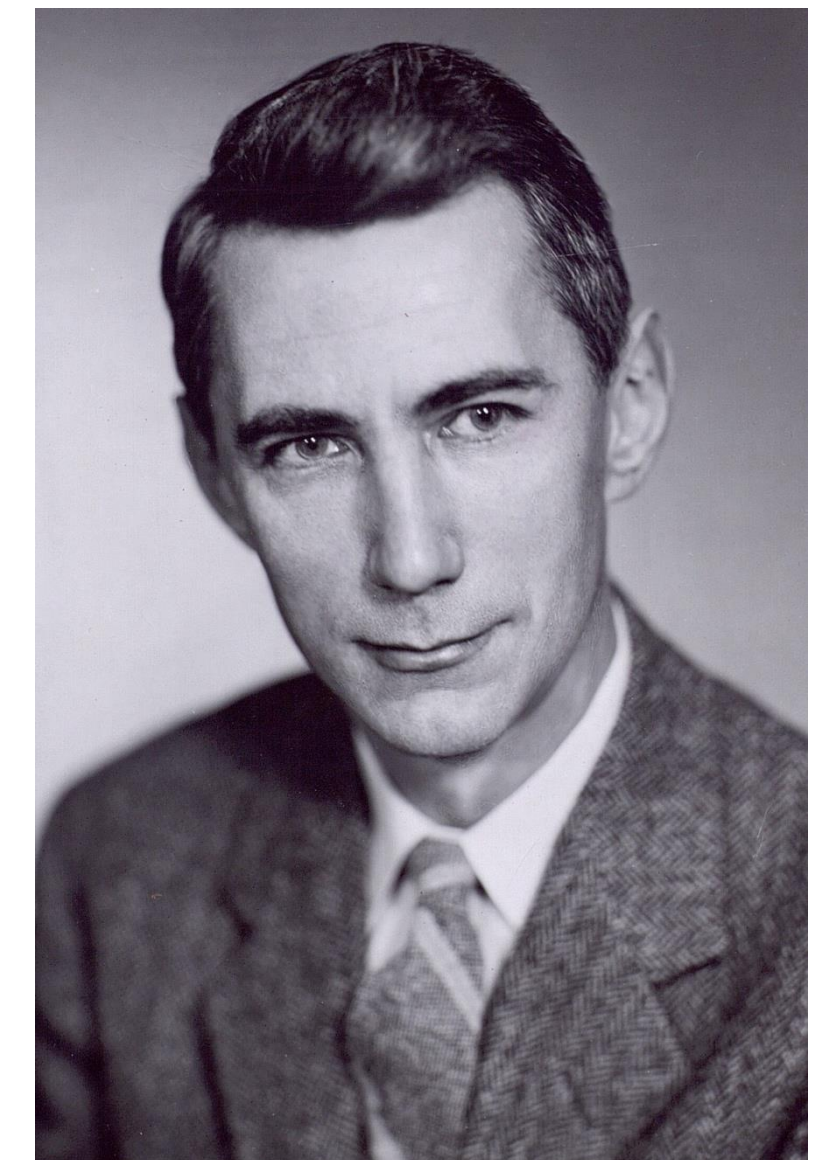
- **natural language processing (NLP)** sometimes is further divided into two directions
 - **natural language understanding (NLU)**
 - **natural language generation (NLG)**: a subset of NLP to convert raw data into coherent narratives or responses
 - a weather forecast written from a table of numbers
 - a summary of a long article
 - the reply of a chatbot

natural language generation

- so far a language model has scored a sentence
 - *she was not afraid* beats *afraid not was she*
- and predicted the next word
 - after *so*, *much*
- a language model also can **generate**
 - predict, output the word, predict again
- we'll start from a method dating back to 1948
 - hold your horses for LLM until week 12

Shannon, 1948

- the first text generator: a book and a thumb
 - open the book at random, write down the word you land on
 - open another page, read until that word turns up, write down the word after it
 - repeat



Claude Shannon

Shannon, 1948

examples

- each word chosen by its own frequency
 - REPRESENTING AND SPEEDILY IS AN GOOD APT OR COME CAN
DIFFERENT NATURAL HERE HE THE A IN CAME THE TO OF TO
EXPERT GRAY COME TO FURNISHES THE LINE MESSAGE HAD BE
THESE.
- each word chosen by the word before it
 - THE HEAD AND IN FRONTAL ATTACK ON AN ENGLISH WRITER THAT
THE CHARACTER OF THIS POINT IS THEREFORE ANOTHER METHOD
FOR THE LETTERS THAT THE TIME OF WHO EVER TOLD THE
PROBLEM FOR AN UNEXPECTED.

Sherlock Holmes

- *The Adventures of Sherlock Holmes* by Arthur Conan Doyle has 121,014 tokens
- after *the*
 - door 88, matter 83, other 80, house 63, room 61
- after *said*
 - he 140, holmes 110, i 52
- after *sherlock*
 - holmes, 91 times out of 97





after	next word, count (die faces)
sherlock	holmes 91 (1–6)
holmes	had 16 (1–3), was 13 (4–6)
had	a 36 (1–4), the 18 (5–6)
was	a 160 (1–4), the 61 (5–6)
a	little 99 (1–3), man 80 (4–6)
little	man 4 (1–6)
man	who 38 (1–5), was 6 (6)
who	had 35 (1–4), was 23 (5–6)
the	door 88 (1–4), man 44 (5–6)
door	and 13 (1–4), was 6 (5–6)
and	i 208 (1–3), the 199 (4–6)
i	have 301 (1–3), was 186 (4–5), had 168 (6)
have	a 47 (1–3), the 30 (4–5), had 24 (6)

activity 1

- start with *sherlock*
- add one word at a time
 - option 1: take the most probable next word (next word with highest count)
 - option 2: roll the die, in the notebook or on your phone, and take the word whose faces came up
- stop when you have a total of eight words

activity 1

debrief

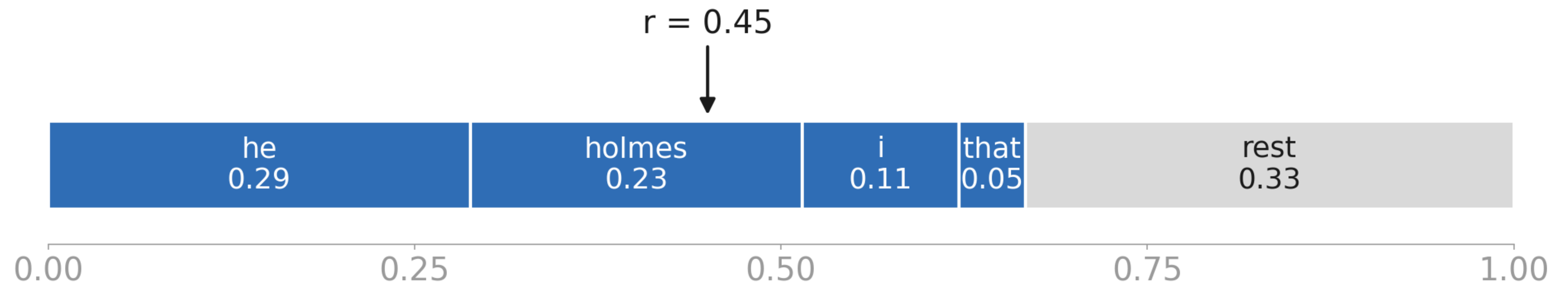
- option 1: *sherlock holmes had a little man who had*
- option 2: could be different sentences
- **decoding**: turning a table of words with probabilities into sentences
- **greedy decoding**: choosing the most probable item every time
- **sampling**: choosing based on probability



sampling

sampling

- a line from 0 to 1: each word owns a stretch as wide as its probability
- draw a random number: the stretch it lands in is the word you write



sampling

step	after	stretches on the line	r	drawn
1	sherlock	holmes 0.00–0.94, holmes' 0.94–1	0.35	holmes
2	holmes	, 0.00–0.33, . 0.33–0.51, had 0.51–0.55, rest 0.55–1	0.51	had
3	had	been 0.00–0.14, a 0.14–0.18, no 0.18–0.22, rest 0.22–1	0.04	been
4	been	a 0.00–0.05, in 0.05–0.07, of 0.07–0.09, rest 0.09–1	0.08	of
5	of	the 0.00–0.27, a 0.27–0.33, his 0.33–0.38, rest 0.38–1	0.02	the
6	the	door 0.00–0.02, matter 0.02–0.03, other 0.03–0.04, rest 0.04–1	0.88	condition

sampling

- in Python, `random.choices` draws from a list with weights
- the weights need not add up to 1, so the counts work as they are

```
row = holmes_cfd["said"]  
words = list(row)  
weights = [row[w] for w in words]  
random.choices(words, weights=weights)[0]  
→ 'he'
```

generate

- one row at a time: look up the row of the last word, draw, append, repeat

```
def generate(model, start, n=15):  
    ...     words = [start]  
    ...     while len(words) < n:  
    ...         words.append(sample(model[words[-1]]))  
    ...     return " ".join(words)  
generate(holmes_cfd, "the")
```

→ the table . the coroner asked in the king had forgotten the brandy and see

greedy decoding

- the biggest count every time, so the same start always gives the same text

```
def greedy(model, start, n=15):  
...     words = [start]  
...     while len(words) < n:  
...         words.append(model[words[-1]].max())  
...     return " ".join(words)  
greedy(holmes_cfd, "the")
```

→ the door , and i have been a little more than i have been a



n-gram (revisit)

n-gram

unigram vs. bigram

- if we generate from a unigram language model based on Shakespeare
 - *To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have*
- for a bigram model
 - *Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow.*

n-gram

trigram & 4-gram

- trigram
 - *Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done.*
- 4-gram
 - *King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in;*
 - *It cannot be but so.*
- *It cannot be but so* is a line of *King John*

n-gram memorization

- for the Shakespeare text
 - $|V| = 29,066$
 - possible bigrams = $|V|^2 = 844$ million
 - possible 4-grams = 7×10^{17}
- but not all are attested
 - most 4-grams occur once in Shakespeare
 - after *It cannot be but*, only seven words ever follow (*that, I, he, thou, so, and*)
 - when the language model has no choice, it can output from memorization

n-gram

genre

- language models reflect the genre of their training corpus
- a trigram output from the WSJ
 - *They also point to ninety nine point six billion dollars from two hundred four oh six three percent of the rates of interest stores as Mexico and Brazil on market conditions*
- it is still English data
 - but we can tell the training corpus is not Shakespeare

activity 2

- in the notebook
 - pick a novel: *Alice in Wonderland*, *Pride and Prejudice*, the complete Shakespeare, or the *Odyssey*
- train a bigram model and generate five sentences starting with *the*
- paste a sentence into the Pollev **without the title**
 - pollev.com/ling430
- let us guess which book is used

lec07a activity 2

Nobody has responded yet.
Hang tight! Responses are coming in.



perplexity (revisit)

perplexity

- suppose a model has learned three words: *red green blue*
- and you have a text: *red red red red blue*
- perplexity: the inverse probability of the text, normalized by its length N

$$PP(\text{text}) = P(\text{text})^{-1/N}$$

- a model that learned nothing: each word 1/3

$$P(\text{text}) = \left(\frac{1}{3}\right)^5 = 0.0041$$

$$PP(\text{text}) = 0.0041^{-1/5} = 3$$

- a model that learned *red* dominates: $P(\text{red}) = 0.8$, the others 0.1

$$P(\text{text}) = 0.8^4 \times 0.1 = 0.041$$

$$PP(\text{text}) = 0.041^{-1/5} = 1.89$$

perplexity

- perplexity measures the **match to the training text**, not the quality of the sentence
- useful for comparing two models on the same test text

sentence	perplexity under the Sherlock Holmes unigram model
the the the the the	22
the holmes said the man	105
she was not afraid	404
my laptop crashed yesterday	infinite: laptop is not in the table



temperature

softmax

- a large language model scores every word; a score can be negative
- softmax: exponentiate each score, then divide by the total

$$p(w) = \exp(z_w) / \sum_v \exp(z_v)$$

word	score z	exp(z)	p
all	1.2	3.32	0.44
the	0.9	2.46	0.33
your	0.1	1.11	0.15
that	-0.5	0.61	0.08
total		7.49	1.00

temperature

- we've seen two ways to decode
 - greedy: safe, but can repeat itself
 - sampling: varied, but can be gibberish
- temperature: a dial between the two

temperature

- temperature can be applied to the softmax output

next word	score	$\tau = 1$	$\tau = 0.5$	$\tau = 0.1$	$\tau = 10$
all	1.2	0.44	0.59	0.95	0.27
the	0.9	0.33	0.32	0.05	0.26
your	0.1	0.15	0.07	0.00	0.24
that	-0.5	0.08	0.02	0.00	0.23

- temperature** τ divides every score before the softmax
 - low τ sharpens the menu toward the top word
 - high τ flattens it toward a fair die
- $\tau \rightarrow 0$ is greedy decoding; $\tau \rightarrow \infty$ is random sampling

temperature

- three runs from *the* at three different temperatures
- $\tau = 0.5$
 - *the house , and i have the pool , and so as to her ,*
- $\tau = 1$
 - *the evidence . he could not think over you say , a scent , where*
- $\tau = 2$
 - *the interest in 'frisco , still open , beckoning to america they came over utterly*

top-k & top-p

- two more knobs cut off the tail of the menu
- **top-k sampling**: keep the k most probable words, renormalize, sample; $k = 1$ is greedy
- **top-p sampling**: keep the smallest set of words whose probabilities add up to p , e.g. 0.9, then sample
 - the set grows when the model is unsure, shrinks when it is confident
- they are used to make chatbots more “creative” or “precise”



the tweet generator

the corpus

- @realDonaldTrump, Jan 2017 to Jan 2021: 10,471 tweets, 204,607 tokens, retweets and links removed, one tweet per line
- each tweet gets `<s>` and `</s>`, so the model knows how a tweet starts and ends
- capitalization kept this time
 - they are part of this person's style

first word	tweets
The	1,185
I	705
Thank	304
Just	259
We	257
Great	231

activity 3

- upload the tweets_trump.txt from Canvas
- generate ten tweets at $\tau = 1$
- then try different settings at $\tau = 0.5$ and ten at $\tau = 2$
- pick the most convincing fake and the most broken one
- check the convincing one against the corpus: is it a real tweet, word for word?

activity 3

debrief

- greedy
 - *The Democrats are doing a great honor to the United States .*
- $\tau = 0.5$
 - *Just spoke to be going to congratulate him that the U .*
- $\tau = 1$
 - *The Democrats are refusing scripted question ! 54*
 - *CHINESE VIRUS FACT CHECK*
- $\tau = 2$
 - *Venezuela stand stronger America Safe*

since 1948

- what is unchanged
 - models still do the task of predicting the next word and sampling with a temperature
- what has changed
 - statistical and neural methods have replaced a count table of words
 - the context grew from a few words in n-grams to be much longer

next time

- Thursday
 - text classification with Naive Bayes
 - Jurafsky & Martin, Appendix B, §§B.1–B.3
 - pandas: What Kind of Data Does pandas Handle?
- **HW 4 to be released Thursday**